

C# Tutorial

This tutorial covers Microsoft's new programming language C#. It is aimed at people with some programming experience, although it tries not to assume too much knowledge. The [contents](#) page contains links to all the lessons, and to subsections of these lessons. Otherwise, choose from the links in the box on the right.

Note that the tutorial covers the syntax of C#, rather than going through the .NET framework classes. Since there are over 6000 of these classes, this would have been rather a big job. Indeed, the coverage of the syntax as it is isn't anywhere near exhaustive (for detailed specifications, see the C# reference material).

If you have any comments, errata or questions about these lessons, please get in touch with our C# pro at andy@softsteel.co.uk. Note, though, that it is not his job to teach anyone programming, or to do anyone's homework.

* 25/04/03 - thanks to more feedback the enum example now shows the right output and we've hopefully stopped the occasional use of 'chapter' for 'lesson'.

* 17/04/03 - we're grateful to a correspondent for putting us right about the 'param' keyword discussed in lesson 13. Method signatures don't include the params keyword, but they do include the parameters marked by this keyword.

* 05/02/03 - someone discovered our mistake in the code in lesson 10, so we put it right. In which lesson will the next error be found? Why not get together with friends and play bug sweepstake.

* 07/03/02 - we've rewritten the lesson on xml documentation (lesson 19). As well as being more thorough, it raises an issue about conflicting advice on how to use some of the tags, and highlights the many problems with the documenter. We also unveil the Softsteel C# Documenter, a free software project released under the GNU GPL.

* 12/02/02 - with the final release of the .NET framework we have been through and checked all the code, fixing various errata. Also added the 'patchwork book', a collection of links to sample chapters available on the web.

* 06/02/02 - corrected one erratum, added lots of books to the list of C# books

* 04/01/02 - corrected some errata about pointers on lesson 5 (pointed out by another helpful correspondent), and added some explanatory text

* 02/12/01 - corrected some errata on lesson 13 (pointed out by a helpful correspondent)

* 07/10/01 - following feedback, we have added a section on indexers. In response to other queries, we should also like to say that we are open to the idea of covering the .NET classes by accepting other people's tutorials. We would review and verify these, and retain the right to rewrite or expand them to fit our style, but the original author would be fully credited. (If this is too much of an imposition, there are plenty of other C# sites out there which host tutorials).

* 09/09/01 - a few updates have been made to account for beta 2 release changes.

* We have now set up links to recommended C# books available at Amazon.

* Following feedback, we have now introduced printer-friendly versions of these lessons. Note, however, that we have no plans to publish a single document containing the whole tutorial.

Lesson 1: Introducing the Microsoft .NET Framework

.NET (dot-net) is the name Microsoft gives to its general vision of the future of computing, the view being of a world in which many applications run in a distributed manner across the Internet. We can identify a number of different motivations driving this vision.

Firstly, distributed computing is rather like object oriented programming, in that it encourages specialized code to be collected in one place, rather than copied redundantly in lots of places. There are thus potential efficiency gains to be made in moving to the distributed model.

Secondly, by collecting specialized code in one place and opening up a generally accessible interface to it, different types of machines (phones, handhelds, desktops, etc.) can all be supported with the same code. Hence Microsoft's 'run-anywhere' aspiration.

Thirdly, by controlling real-time access to some of the distributed nodes (especially those concerning authentication), companies like Microsoft can control more easily the running of its applications. It moves applications further into the area of 'services provided' rather than 'objects owned'.

Interestingly, in taking on the .NET vision, Microsoft seems to have given up some of its proprietary tendencies (whereby all the technology it touched was warped towards its Windows operating system). Because it sees its future as providing software services in distributed applications, the .NET framework has been written so that applications on other platforms will be able to access these services. For example, .NET has been built upon open standard technologies like XML and SOAP.

At the development end of the .NET vision is the .NET Framework. This contains the Common Language Runtime, the .NET Framework Classes, and higher-level features like ASP.NET (the next generation of Active Server Pages technologies) and WinForms (for developing desktop applications).

The Common Language Runtime (CLR) manages the execution of code compiled for the .NET platform. The CLR has two interesting features. Firstly, its specification has been opened up so that it can be ported to non-Windows platforms. Secondly, any number of different languages can be used to manipulate the .NET framework classes, and the CLR will support them. This has led one commentator to claim that under .NET the language one uses is a 'lifestyle choice'.

Not all of the supported languages fit entirely neatly into the .NET framework, however (in some cases the fit has been somewhat Procrustean). But the one language that is guaranteed to fit in perfectly is C#. This new language, a successor to C++, has been released in conjunction with the .NET framework, and is likely to be the language of choice for many developers working on .NET applications.

For more information about .NET, see our [tutorial](#), or the reference section ([lesson 20](#)).

Lesson 2: Comparing C# to C++ and Java

This lesson gives a brief overview of the differences between C# and the two languages that are its closest relatives. References are given in each case to more comprehensive works currently to be found on the web.

C# versus Java

C# and Java are both new-generation languages descended from a line including C and C++. Each includes advanced features, like garbage collection, which remove some of the low level maintenance tasks from the programmer. In a lot of areas they are syntactically similar.

Both C# and Java compile initially to an intermediate language: C# to Microsoft Intermediate Language (MSIL), and Java to Java bytecode. In each case the intermediate language can be run - by interpretation or just-in-time compilation - on an appropriate 'virtual machine'. In C#, however, more support is given for the further compilation of the intermediate language code into native code.

C# contains more primitive data types than Java ([lesson 4](#)), and also allows more extension to the value types. For example, C# supports 'enumerators', types which are limited to a defined set of constant variables ([lesson 7](#)), and 'structs', which are user-defined value types ([lesson 11](#)).

Unlike Java, C# has the useful feature that we can overload various operators.

Like Java, C# gives up on multiple class inheritance in favour of a single inheritance model extended by the multiple inheritance of interfaces ([lesson 11](#)). However, polymorphism ([lesson 14](#)) is handled in a more complicated fashion; with base class methods either 'overriding' or 'hiding' super class methods

C# also uses 'delegates' - type-safe method pointers (see [lesson 16](#)). These are used to implement event-handling.

In Java, multi-dimensional arrays are implemented solely with single-dimensional arrays (where arrays can be members of other arrays. In addition to jagged arrays, however, C# also implements genuine rectangular arrays ([lesson 6](#)).

For more comparison of C# and Java see:

[A Comparative Overview of C#](#)

[Microsoft .NET vs J2EE: How do they stack up?](#)

C# versus C++

Although it has some elements derived from Visual Basic and Java, C++ is C#'s closest relative.

In an important change from C++, C# code does not require header files. All code is written inline.

As touched on above, the .NET runtime in which C# runs performs memory management, taking care of tasks like garbage collection. Because of this, the use of pointers in C# is much less important than in C++. Pointers can be used in C#, where the code is marked as 'unsafe'

([lesson 5](#)), but they are only really useful in situations where performance gains are at an absolute premium.

Speaking generally, the 'plumbing' of C# types is different from that of C++ types, with all C# types being ultimately derived from the 'object' type ([lesson 4](#)). There are also specific differences in the way that certain common types can be used. For instance, C# arrays are bounds checked unlike in C++, and it is therefore not possible to write past the end of a C# array.

C# statements are quite similar to C++ statements. To note just one example of a difference: the 'switch' statement has been changed so that 'fall-through' behaviour is disallowed ([lesson 10](#)).

As mentioned above, C# gives up on the idea of multiple class inheritance. Other differences relating to the use of classes are: there is support for class 'properties' of the kind found in Visual Basic, and class methods are called using the . operator rather than the :: operator.

For more comparison of C# and C++ see:

[C++ -> C#: What you need to know to move from C++ to C#.](#)

[Deep Inside C#: An Interview with Microsoft Chief Architect Anders Hejlsberg.](#)

Lesson 3: Getting Started

In order to use C# and the .NET framework classes, you first need to install either the .NET framework SDK, or else Visual Studio .NET. Some useful advice about getting hold of and installing the former can be found at:

<http://www.mastercsharp.com/article.aspx?ArticleID=17&TopicID=10>

In the next section we run through a standard 'hello world' example, with links to lessons covering the different parts of the program.

A First C# Program: 'Hello World'

Let's begin in the traditional way, by looking at the code of a Hello World program (note that the tabulation and line numbers are included just for the sake of readability).

```
1.    using System;
2.    public class HelloWorld
3.    {
4.        public static void Main()
5.        {
6.            // This is a single line comment
7.            /* This is a
8.               Multiple
9.               line comment */
10.         Console.WriteLine("Hello World! From Softsteel
              Solutions");
11.     }
12. }
```

The first thing to note about C# is that it is case-sensitive. You will therefore get compiler errors if, for instance, you write 'console' rather than 'Console'.

The second thing to note is that every statement finishes with a semicolon (;) or else takes a code block within curly braces.

As C# is an object-oriented language, C# programs must be placed in classes (classes are discussed in [lesson 11](#), but if you are new to object orientation we suggest that you first read some introductory material). Line 2 above declares the class to be named 'Hello World'.

Line 1 of the code declares we are using the System namespace (namespaces are also

covered in [lesson 11](#)). The point of this declaration is mostly to save ourselves time typing. Because the 'Console' object used in line 10 of the code actually belongs to the 'System' namespace, its fully qualified name is 'System.Console'. However, because in line 1 we declare that the code is using the System namespace, we can then leave off the 'System.' part of its name within the code.

When compiled and run, the program above will automatically run the 'Main' method declared and begun in line 4. Note again C#'s case-sensitivity - the method is 'Main' rather than 'main'.

Lines 6-9 of the program are ignored by the compiler, being comments entered by the programmer for his own benefit. Line 6 shows a single line comment, in which everything on the line after the two forward slashes is ignored by the compiler. Lines 7-9 demonstrate a multi-line comment, in which everything between the opening `/*` and closing `*/` is ignored, even when it spans multiple lines.

The statement on line 10 calls the 'WriteLine' method of the Console class in the System namespace. It should be obvious how this works in the given example - it just prints out the given string to the 'Console' (on PC machines this will be a DOS prompt). For a more complicated use of the WriteLine method, see [lesson 7](#).

In order to run it, the program above must first be saved in a file. Unlike in Java, the name of the class and the name of the file in which it is saved do not need to match up, although it does make things easier if you use this convention. In addition, you are free to choose any extension for the file, but it is usual to use the extension '.cs'.

Suppose that you have saved the file as 'HelloWorld.cs'. Then to compile the program from a command line, you would use the command

```
csc HelloWorld.cs
```

(for Visual Studio .NET users: compile by pressing Ctrl-Shift-B)

This command would generate the executable HelloWorld.exe, which could be run in the usual way, by entering its name:

```
HelloWorld
```

(for Visual Studio .NET users: run by pressing Ctrl-F5)

Fairly obviously, this program would produce the output:

```
Hello World! From Softsteel Solutions.
```

Lesson 4: Variable Types (1): Reference Types and Value Types

C# is a type-safe language. Variables are declared as being of a particular type, and each variable is constrained to hold only values of its declared type.

Variables can hold either value types or reference types, or they can be pointers. This lesson covers the first two options; pointers are discussed in [lesson 5](#).

Here's a quick recap of the difference between value types and reference types.

- where a variable *v* contains a value type, it directly contains an object with some value. No other variable *v'* can directly contain the object contained by *v* (although *v'* might contain an object with the same value).
- where a variable *v* contains a reference type, what it directly contains is something which refers to an object. Another variable *v'* can contain a reference to the same object referred to by *v*.

Value Types

It is possible in C# to define your own value types by declaring enumerators ([lesson 7](#)) or structs ([lesson 11](#)). These user-defined types are mostly treated in exactly the same way as C#'s predefined value types, although compilers are optimized for the latter. The following table lists, and gives information about, the predefined value types. Because in C# all of the apparently fundamental value types are in fact built up from the (actually fundamental) object type, the list also indicates which System types in the .Net framework correspond to these pre-defined types.

C# Type	.Net Framework (System) type	Signed?	Bytes Occupied	Possible Values
sbyte	System.Sbyte	Yes	1	-128 to 127
short	System.Int16	Yes	2	-32768 to 32767
int	System.Int32	Yes	4	-2147483648 to 2147483647
long	System.Int64	Yes	8	-9223372036854775808 to 9223372036854775807
byte	System.Byte	No	1	0 to 255
ushort	System.UInt16	No	2	0 to 65535
uint	System.UInt32	No	4	0 to 4294967295
ulong	System.UInt64	No	8	0 to 18446744073709551615
float	System.Single	Yes	4	Approximately $\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$ with 7 significant figures
double	System.Double	Yes	8	Approximately $\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$ with 15 or 16 significant figures

decimal	System.Decimal	Yes	12	Approximately $\pm 1.0 \times 10^{-28}$ to $\pm 7.9 \times 10^{28}$ with 28 or 29 significant figures
char	System.Char	N/A	2	Any Unicode character (16 bit)
bool	System.Boolean	N/A	1 / 2	true or false

In the following lines of code, two variables are declared and set with integer values.

```
int          x          =          10;
int          y          =          x;
y = 20; // after this statement x holds value 10 and y holds value 20
```

Reference Types

The pre-defined reference types are object and string, where object - as we have mentioned above - is the ultimate base class of all other types. New reference types can be defined using 'class', 'interface', and 'delegate' declarations (covered in [lesson 12](#)).

Reference types actually hold the value of a memory address occupied by the object they reference. Consider the following piece of code, in which two variables are given a reference to the same object (for the sake of the example, this object is taken to contain the numeric property 'myValue').

```
object          x          =          new          object();
x.myValue
object          y          =          x;
y.myValue = 20; // after this statement both x.myValue and y.myValue equal
20
```

This code illustrates how changing a property of an object using a particular reference to it is reflected in all other references to it. Note, however, that although strings are reference types, they work rather more like value types. When one string is set to the value of another, eg

```
string          s1          =          "hello";
string s2 = s1;
```

Then s2 does at this point reference the same string object as s1. However, when the value of s1 is changed, for instance with

```
s1 = "goodbye";
```

what happens is that a new string object is created for s1 to point to. Hence, following this piece of code, s1 equals "goodbye", whereas s2 still equals "hello".

The reason for this behaviour is that string objects are 'immutable'. That is, the properties of these objects can't themselves change. So in order to change what a string variable references, a new string object must be created.

Escape Sequences and Verbatim Strings

When declaring a string variable, certain characters can't, for various reasons, be included in the usual way. C# supports two different solutions to this problem.

The first approach is to use 'escape sequences'. For example, suppose that we want to set variable a to the value:

```
"Hello                                     World
How are you"
```

We could declare this using the following command, which contains escape sequences for the quotation marks and the line break.

```
string a = "\"Hello World\nHow are you\"";
```

The following table gives a list of the escape sequences for the characters that can be escaped in this way:

Character	Escape Sequence
'	\'
"	\"
\	\\
Alert	\a
Backspace	\b
Form feed	\f
New Line	\n
Carriage Return	\r
Horizontal Tab	\t
Vertical Tab	\v
A unicode character specified by its number e.g. \u200	\u
A unicode character specified by its hexadecimal code e.g. \xc8	\x
null	\0 (zero)

The second approach is to use 'verbatim string' literals. These are defined by enclosing the required string in the characters "@" and ". To illustrate this, to set the variable 'path' to the following value:

```
C:\My Documents\
```

we could either escape the back-slash characters

```
string path = "C:\\My Documents\\"
```

or use a verbatim string thus:

```
string path = @"C:\MyDocuments\"
```

Usefully, strings written using the verbatim string syntax can span multiple lines, and whitespace is preserved. The only character that needs escaping is the double-quote

character, the escape sequence for which is two double-quotes together. For instance, suppose that you want to set the variable 'text' to the following value:

the word "big" contains three letters.

Using the verbatim string syntax, the command would look like this:

```
string text = @"the word ""big"" contains three letters."
```

Boxing

C# allows you convert any value type to a corresponding reference type, and to convert the resultant 'boxed' type back again. The following piece of code demonstrates boxing. When the second line executes, an object is initiated as the value of 'box', and the value held by i is copied across to this object. It is interesting to note that the runtime type of box is returned as the boxed value type; the 'is' operator thus returns the type of box below as 'int'.

```
int i = 123;
object box = i;
if (box is int)
{Console.WriteLine("Box contains an int");} // this line is printed
```

Lesson 5: Variable Types(2): Pointers

This lesson gives a brief overview of pointers and their use in C#. It only scratches the surface of a complicated topic, however, so if you are new to pointers it is recommended that you do further reading before using them in your code. Luckily, pointers are only really needed in C# where execution speed is highly important.

Pointer Notation

A pointer is a variable that holds the memory address of another type. In C#, pointers can only be declared to hold the memory addresses of value types (except in the case of arrays - see below).

Pointers are declared implicitly, using the 'dereferencer' symbol *, as in the following example:

```
int *p;
```

[Note that some coders place the dereferencer symbol immediately after the type name, eg.

```
int* p;
```

This variation appears to work just as well as the previous one.]

This declaration sets up a pointer 'p', which will point to the initial memory address of an integer (stored in four bytes).

The combined syntactical element *p ('p' prefixed by the dereferencer symbol '*') is used to refer to the type located at the memory location held by p. Hence given its declaration, *p can appear in integer assignments like the following:

```
*p = 5;
```

This code gives the value 5 to the integer that was initialized by the declaration. It is important, however, not to confuse such an assignment with one in which the dereferencer symbol is absent, e.g.

```
p = 5;
```

The effect of this assignment is to change the memory location held by p. It doesn't change the value of the integer initialised by the original declaration; it just means that p no longer points to that integer. In fact, p will now point to the start of the four bytes present at memory location 5.

Another important symbol for using pointers is the operator &, which in this context returns the memory address of the variable it prefixes. To give an example of this symbol, the following code sets up p to point to integer i's memory location:

```
int i = 5;
int *p;
p = &i;
```

Given the above, the code

```
*p = 10;
```

changes the value of i to 10, since '*p' can be read as 'the integer located at the memory value held by p'.

There is another important piece of notation for pointers. Pointers can be declared for structs (see [lesson 11](#)), as in the following example (which uses the 'Coords' struct defined further below):

```
Coords x = new Coords();  
Coords *y = &x;
```

One can then use the declared pointer y to access a public field of x (say z). This would be done using either the expression

```
(*y).z
```

or the equivalent expression, which uses the -> string:

```
y -> x
```

Unsafe Code

A major problem with using pointers in C# is that C# operates a background garbage collection process. In freeing up memory, this garbage collection is liable to change the memory location of a current object without warning. So any pointer which previously pointed to that object will no longer do so. Such a scenario leads to two potential problems. Firstly, it could compromise the running of the C# program itself. Secondly, it could affect the integrity of other programs.

Because of these problems, the use of pointers is restricted to code which is explicitly marked by the programmer as 'unsafe'. Because of the potential for malicious use of unsafe code, programs which contain unsafe code will only run if they have been given full trust.

To address the problem of garbage collection, one can declare a pointer within a 'fixed' expression. This 'pins' the location of the type pointed to - the memory location of the type therefore remains static, safe from garbage collection. Note that the fixed statement can only be used within the context of unsafe code.

There is a further quirk to learn. Any value types declared within unsafe code are automatically 'fixed', and will generate compile-time errors if used within fixed expressions. The same is not true of reference types, however (for the difference between value and reference types see [lesson 4](#)).

The following code gives an example of a method marked 'unsafe'. From the previous paragraph it follows that the pointer p cannot be declared within a 'fixed' statement on line 9, because p is set up to point to the struct c (a value type) which is declared within the unsafe code

```
1. using System;  
2. public struct Coords
```

```

3.  {
4.      int x;
5.      int y;
6.      Unsafe public static void Main()
7.      {
8.          Coords c = new Coords();
9.          Coords *p = &c;
10.         {
11.             p->y = 6;
12.             (*p).x = 5;
13.         }
14.         Console.WriteLine(c.y);
15.         Console.WriteLine(c.x);
16.     }
17. }

```

Compare this with the following code, in which the pointer p on line 8 must be declared within a 'fixed' statement, because it is set up to point to a type which is not declared within the unsafe block of code:

```

1.  using System;
2.  public struct Coords
3.  {
4.      int x;
5.      int y;
6.      Unsafe public static void notMain(ref Coords c)
7.      {
8.          fixed (Coords *p = &c)
9.          {
10.             p->y = 6;
11.             (*p).x = 5;
12.         }
13.         Console.WriteLine(c.y);
14.         Console.WriteLine(c.x);
15.     }
16. }

```

In the examples given above, 'unsafe' is included as a method modifier. However, it can also be used within a code block, as in the following code fragment:

```

1.    using System;
2.    public static void Main()
3.    {
4.        unsafe
5.        {
6.            Coords c = new Coords();
7.            [...]
8.        }
9.    }

```

Pointers, Methods and Arrays

Although we stated above that pointers can only be used with value types, an exception to this involves arrays (some authors state that the same exception applies to strings, but we have never been able to make this work).

A pointer can be declared in relation to an array, as in the following:

```

int[]      a      =      {4,      5};
int *b = a;

```

What happens in this case is that the memory location held by b is the location of the first type held by a. This first type must, as before, be a value type. The code beneath shows that it is possible to step through the values of an array using a pointer, but explaining this further goes beyond the scope of this tutorial.

```

1.    using System;
2.    public class Tester
3.    {
4.        public static void Main()
5.        {
6.            int[] a = {4, 5};
7.            changeVal(a);
8.            Console.WriteLine(a[0]);
9.            Console.WriteLine(a[1]);
10.       }
11.
12.       public unsafe static void changeVal(int[] a)
13.       {
14.           fixed (int *b = a)
15.           {
16.               *b = 5;
17.               *(b + 1) = 7;

```

18. }

19. }

20. }

Lesson 6: Arrays

Single-Dimensional Arrays

The type of each array declared is given firstly by the type of basic elements it can hold, and secondly by the number of dimensions it has. Single-dimensional arrays have a single dimension (ie, are of rank 1). They are declared using square brackets, eg:

```
int[] i = new int[100];
```

This line of code declares variable `i` to be an integer array of size 100. It contains space for 100 integer elements, ranging from `i[0]` to `i[99]`.

To populate an array one can simply specify values for each element, as in the following code:

```
int i = new int[2];  
i[0] = 1;  
i[1] = 2;
```

One can also run together the array declaration with the assignment of values to elements using

```
int[] i = new int[] {1,2};
```

or the even shorter version of this:

```
int[] i = {1,2};
```

By default, as we have seen, all arrays start with their lower bound as 0 (and we would recommend that you stick with this default). However, using the .NET framework's `System.Array` class it is possible to create and manipulate arrays with an alternative initial lower bound.

The (read-only) `Length` property of an array holds the total number of its elements across all of its dimensions. As single-dimensional arrays have just one dimension, this property will hold the length of the single dimension. For instance, given the definition of array `i` above, `i.Length` is 2.

Rectangular Arrays

C# supports two types of multidimensional arrays: rectangular and jagged. A rectangular array is a single array with more than one dimension, with the dimensions' sizes fixed in the array's declaration. The following code creates a 2 by 3 multi-dimensional array:

```
int[,] squareArray = new int[2,3];
```

As with single-dimensional arrays, rectangular arrays can be filled at the time they are declared. For instance, the code

```
int[,] squareArray = {{1, 2, 3}, {4, 5, 6}};
```

creates a 2 by 3 array with the given values. It is, of course, important that the given values do fill out exactly a rectangular array.

The `System.Array` class includes a number of methods for determining the size and bounds of arrays. These include the methods `GetUpperBound(int i)` and `GetLowerBound(int i)`, which return, respectively, the upper and lower subscripts of dimension `i` of the array (note that `i` is zero based, so the first array is actually array 0).

For instance, since the length of the second dimension of `squareArray` is 3, the expression

```
squareArray.GetLowerBound(1)
```

returns 0, and the expression

```
squareArray.GetUpperBound(1)
```

returns 2.

`System.Array` also includes the method `GetLength(int i)`, which returns the number of elements in the `i`th dimension (again, zero based).

The following piece of code loops through `squareArray` and writes out the value of its elements (loops are covered in [lesson 9](#)).

```
1.    for(int i = 0; i < squareArray.GetLength(0); i++)
2.        for (int j = 0; j < squareArray.GetLength(1); j++)
3.            Console.WriteLine(squareArray[i,j]);
```

A `foreach` loop can also be used to access each of the elements of an array in turn, but using this construction one doesn't have the same control over the order in which the elements are accessed.

Jagged Arrays

Using jagged arrays, one can create multidimensional arrays with irregular dimensions. This flexibility derives from the fact that multidimensional arrays are implemented as arrays of arrays. The following piece of code demonstrates how one might declare an array made up of a group of 4 and a group of 6 elements:

```
int[][] jag = new int[2][];
jag[0] = new int[4];
jag[1] = new int[6];
```

The code reveals that each of `jag[0]` and `jag[1]` holds a reference to a single-dimensional `int` array. To illustrate how one accesses the integer elements: the term `jag[0][1]` provides access to the second element of the first group.

To initialise a jagged array whilst assigning values to its elements, one can use code like the following:

```
int[][] jag = new int[][] {new int[] {1, 2, 3, 4}, new int[] {5, 6, 7, 8, 9, 10}};
```

Be careful using methods like `GetLowerBound`, `GetUpperBound`, `GetLength`, etc. with jagged arrays. Since jagged arrays are constructed out of single-dimensional arrays, they shouldn't be treated as having multiple dimensions in the same way that rectangular arrays do.

To loop through all the elements of a jagged array one can use code like the following:

```
1.    for (int i = 0; i < jag.GetLength(0); i++)
2.        for (int j = 0; j < jag[i].GetLength(0); j++)
3.            Console.WriteLine(jag[i][j]);
```

or

```
1.    for (int i = 0; i < jag.Length; i++)
2.        for (int j = 0; j < jag[i].Length; j++)
3.            Console.WriteLine(jag[i][j]);
```

Lesson 7: Enumerators

Enumerators are numerical types limited to a restricted set of values. Their specification includes literals which can then be used as constants for the values they are set to. To illustrate such a specification, the following code declares constant values for each of the days of the week.

```
1.  public enum DAYS
2.  {
3.      Monday,
4.      Tuesday,
5.      Wednesday,
6.      Thursday,
7.      Friday,
8.      Saturday,
9.      Sunday
10. }
```

This code is affected by two default settings. Firstly, the default value type of enumerators is integer. Secondly, by default the first literal specified is set to 0 and this value is then incremented for each subsequent literal. Hence, given the above DAYS.Monday equals the integer 0 and DAYS.Sunday equals the integer 6.

The following code illustrates how to force a different numeric type, a byte, for an enumerator:

```
1.  enum byteEnum : byte
2.  {
3.      A,
4.      B
5.  }
```

Furthermore, any and all of the values of the enumerator can be explicitly set. The default incrementation remains in place, so the following code will set Monday to 1 and Sunday to 7.

```
1.  public enum DAYS
2.  {
3.      Monday=1,
4.      Tuesday,
5.      Wednesday,
6.      Thursday,
7.      Friday,
8.      Saturday,
```

```
9.      Sunday
10.   }
```

A useful feature of enumerators is that one can also retrieve the literal as a string from the numeric constant with which it is associated. The following code prints out both the literal and its constant value for the specified enumerator.

```
1.   using System;
2.   public class EnumTest
3.   {
4.       public enum DAYS: byte
5.       {Monday, Tuesday, Wednesday, Thursday, Friday,
        Saturday, Sunday}
6.
7.       public static void Main()
8.       {
9.           Array dayArray =
EnumTest.DAYS.GetValues(typeof(EnumTest.DAYS));
10.          foreach (DAYS day in dayArray)
11.              Console.WriteLine("Number {1} of
EnumTest.DAYS is {0}", day, day.ToString("d"));
12.      }
13.  }
```

(We have had enquiries as to the use of the ToString("d") method in the above, so let's take a second to address these. Originally, using the beta version of the .NET runtime, we used the standard ToString() method. But when we upgraded to the release version, this stopped working, so we came up with this workaround. Of course, in the current version we have installed, ToString() is working again...

The ToString method used above takes a single string parameter, which can be one of "g", "d", "x", "f", or their uppercase equivalents. If you look at the documentation on these you'll see that we should probably be using the default, "g". To be honest, we can't even remember why we used "d", except that it worked. As it's not even clear that it should work, we'll probably switch it back when nobody's looking...)

The output of this code is:

```
Number      0      of      EnumTest.DAYS      is      Monday
Number      1      of      EnumTest.DAYS      is      Tuesday
Number      2      of      EnumTest.DAYS      is      Wednesday
Number      3      of      EnumTest.DAYS      is      Thursday
Number      4      of      EnumTest.DAYS      is      Friday
Number      5      of      EnumTest.DAYS      is      Saturday
Number 6 of EnumTest.DAYS is Sunday
```

[Some explanation is required of the string interpolation in the Console.WriteLine method. What happens here is that any number of objects can be passed into the WriteLine method in

its params array (see [lesson 13](#)). If code {n} is found in the initial string passed to WriteLine, it interpolates into this string the string returned by the 'format' method of the nth extra object passed to it.]

Lesson 8: Operators

C# has a number of standard operators, taken from C, C++ and Java. Most of these should be quite familiar to programmers; the less common ones are covered elsewhere.

The diagram below lists the standard operators. Note that when writing classes it is possible to change the default behaviour of some of these operators (ie to 'overload' the operator), although this should only be done where the resultant semantics makes sense. The diagram indicates which of the operators are overloadable.

Category	Name	Syntax Example	Overloadable?	
Primary	Grouping	(a+b)	No	
	Member	A.B	No	
	Struct pointer member access	A->B	No	
	Method call	f(x)	No	
	Post increment	c++	Yes	
	Post decrement	c--	Yes	
	Constructor call	c = new Coord();	No	
	Array stack allocation	int* c = stackalloc int[10]	No	
	Struct size retrieval	sizeof (int)	No	
	Arithmetic check on	checked {byte c = (byte) d;}	No	
	Arithmetic check off	unchecked {byte c = (byte) d;}	No	
	Unary	Positive value	+10	Yes
		Negative value	-10	Yes
Not		!(c==d)	Yes	
Bitwise complement		~(int x)	Yes	
Pre increment		++c	Yes	
Pre decrement		--c	Yes	
Type cast		(myType)c	No	
Value at address		int* c = d;	No	
Address value of		int* c = &d;	No	
Type operators	Type equality / compatibility	a is String	No	
	Type retrieval	typeof (int)	No	
Arithmetic	Multiplication	c*d	Yes	
	Division	c/d	Yes	

	Remainder	c%d	Yes
	Addition	c+d	Yes
	Subtraction	c-d	Yes
	Shift bits right	c>>3	Yes
	Shift bits left	c<<3	Yes
Relational and Logical	Less than	c<d	Yes
	Greater than	c>d	Yes
	Less than or equal to	c<=d	Yes
	Greater than or equal to	c>=d	Yes
	Equality	c==d	Yes
	Inequality	c!=d	Yes
	Bitwise and	c&d	Yes
	Bitwise or	c d	Yes
	Logical and	c&& d	No
	Logical or	c d	No
	Conditional	int c=(d<10) ? 5:15	No

Overloading operators

To overload an operator in a class, one defines a method using the 'operator' keyword. For instance, the following code overloads the equality operator (see [lesson 13](#) for details about methods).

```
public static bool operator == (Value a, Value b)
{return a.Int == b.Int}
```

Where an operator is one of a logical pair, both operators should be overwritten if any one is. These pairs are the following:

```
== and !=
< and >
<= and >=
```

Lesson 9: Flow Control (1): Loop Statements

C# provides a number of the common loop statements:

- while
- do-while
- for
- foreach

while loops

syntax: while (expression) statement[s]

A 'while' loop executes a statement, or a block of statements wrapped in curly braces, repeatedly until the condition specified by the boolean expression returns false. For instance, the following code

```
1.  int a = 0;
2.  while (a < 3)
3.  {
4.      System.Console.WriteLine(a);
5.      a++;
6.  }
```

produces the following output:

```
0
1
2
```

do-while loops

syntax: do statement[s] while (expression)

A 'do-while' loop is just like a 'while' loop except that the condition is evaluated after the block of code specified in the 'do' clause has been run. So even where the condition is initially false, the block runs once. For instance, the following code outputs '4':

```
1.  int a = 4;
2.  do
3.  {
4.      System.Console.WriteLine(a);
5.      a++;
6.  } while (a < 3);
```

for loops

syntax: `for (statement1; expression; statement2) statement[s]`

The 'for' clause contains three parts. Statement1 is executed before the loop is entered. The loop which is then executed corresponds to the following 'while' loop:

```
statement                                     1
while (expression) {statement[s]; statement2}
```

'For' loops tend to be used when one needs to maintain an iterator value. Usually, as in the following example, the first statement initialises the iterator, the condition evaluates it against an end value, and the second statement changes the iterator value.

```
1.   for (int a =0; a<5; a++)
2.   {
3.       System.Console.WriteLine(a);
4.   }
```

foreach loops

syntax: `foreach (variable1 in variable2) statement[s]`

The 'foreach' loop is used to iterate through the values contained by any object which implements the `IEnumerable` interface. When a 'foreach' loop runs, the given variable1 is set in turn to each value exposed by the object named by variable2. As we have seen previously, such loops can be used to access array values. So, we could loop through the values of an array in the following way:

```
1.   int[] a = new int[]{1,2,3};
2.   foreach (int b in a)
3.       System.Console.WriteLine(b);
```

The main drawback of 'foreach' loops is that each value extracted (held in the given example by the variable 'b') is read-only.

Lesson 10: Flow Control (2): Jump and Selection Statements

Jump Statements

The jump statements include

```
break
continue
goto
return
throw (see lesson 17)
```

(see [lesson 13](#))

break

The 'break' statement breaks out of the 'while' and 'for' loops covered in [lesson 9](#), and the 'switch' statements covered later in this lesson. The following code gives an example - albeit a very inefficient one - of how it could be used. The output of the loop is the numbers from 0 to 5.

```
1.  int a = 0;
2.  while (true)
3.  {
4.      System.Console.WriteLine(a);
5.      a++;
6.      if (a == 5)
7.          break;
8.  }
```

continue

The 'continue' statement can be placed in any loop structure. When it executes, it moves the program counter immediately to the next iteration of the loop. The following code example uses the 'continue' statement to count the number of values between 1 and 100 inclusive that are not multiples of seven. At the end of the loop the variable y holds the required value.

```
1.  int y = 0;
2.  for (int x=1; x<101; x++)
3.  {
4.      if ((x % 7) == 0)
5.          continue;
6.      y++;
7.  }
```

goto

The 'goto' statement is used to make a jump to a particular labelled part of the program code. It is also used in the 'switch' statement described below. We can use a 'goto' statement to construct a loop, as in the following example (but again, this usage is not recommended):

```
1.   int a = 0;
2.   start:
3.   System.Console.WriteLine(a);
4.   a++;
5.   if (a < 5)
6.       goto start;
```

Selection Statements

C# offers two basic types of selection statement:

```
if _____ - _____ else _____
switch - default
```

if - else

'If-else' statements are used to run blocks of code conditionally upon a boolean expression evaluating to true. The 'else' clause, present in the following example, is optional.

```
1.  if (a == 5)
2.      System.Console.WriteLine("A is 5");
3.  else
4.      System.Console.WriteLine("A is not 5");
```

If statements can also be emulated by using the conditional operator. The conditional operator returns one of two values, depending upon the value of a boolean expression. To take a simple example, the line of code

```
int i = (myBoolean) ? 1 : 0 ;
```

sets `i` to 1 if `myBoolean` is true, and sets `i` to 0 if `myBoolean` is false. The 'if' statement in the previous code example could therefore be written like this:

```
1. System.Console.WriteLine( a==5 ? "A is 5" : "A is not 5");
```

switch - default

'Switch' statements provide a clean way of writing multiple if - else statements. In the following example, the variable whose value is in question is 'a'. If a equals 1, then the output is 'a>0'; if a equals 2, then the output is 'a>1 and a>0'. Otherwise, it is reported that the variable is not set.

```

1.  switch(a)
2.  {
3.      case 2:
4.          Console.WriteLine("a>1 and ");
5.          goto case 1;
6.      case 1:
7.          Console.WriteLine("a>0");
8.          break;
9.      default:
10.         Console.WriteLine("a is not set");
11.         break;
12.  }

```

Each case (where this is taken to include the 'default' case) will either have code specifying a conditional action, or no such code. Where a case does have such code, the code must (unless the case is the last one in the switch statement) end with one of the following statements:

```

break;
goto case k; (where k is one of the cases specified)
goto default;

```

From the above it can be seen that C# 'switch' statements lack the default 'fall through' behaviour found in C++ and Java. However, program control does fall through wherever a case fails to specify any action. The following example illustrates this point; the response "a>0" is given when a is either 1 or 2.

```

1.  switch(a)
2.  {
3.      case 1:
4.      case 2:
5.          Console.WriteLine("a>0");
6.          break;
7.      default:
8.          Console.WriteLine("a is not set");
9.          break;
10.  }

```

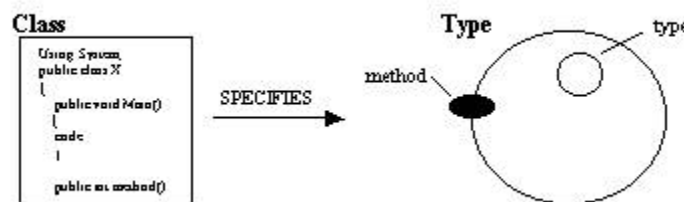
Lesson 11: Introducing Classes, Structs and Namespaces

Classes and Types

As we noted previously, one can create new reference types by defining classes. Classes provide 'templates' from which these direct instances are generated. Where we appeal to the relation between a class and its corresponding reference type instances we shall say that a class specifies the type (also that the class specifies the constitutive elements of the type).

Any type is made up of elements, which we term type members. There are two main kinds of type members that a class can specify. Firstly, a class can specify other types - both value and reference (for the distinction see [lesson 4](#)). This idea, that types can contain other types, is known within the literature on object orientation as 'containment', or else 'aggregation'. Where a type contains another reference type, we shall call it the containing type of the latter.

The second, main kind of type members that a class can specify are methods, functions designed for reading and manipulating the value and reference types an instance contains.



Inheritance

Object oriented languages like C# allow inheritance from reference types. If a type inherits from another, it takes on all of its type members. A type can, however, both add to the members it inherits in this way, as well as 'overwriting' them. To overwrite a type member - a method, say - the defining class specifies a method with the same name as one that it inherits (this is covered in [lesson 14](#)).

C#'s inheritance model is more similar to Java's than to C++'s. In particular, C# classes inherit always from a single base class (if one is not specified in the declaration, inheritance is from System.Object). At the same time, however, C# classes can inherit from any number of interfaces.

Abstract Classes and Interfaces

Some classes are not designed to have direct instances. Rather, they are designed simply to be inherited from, by ancestors which may themselves have direct instances (or not). A class is 'abstract' just in case it cannot itself have direct instances.

Classes can be abstract because in a class it is possible to specify a class method without specifying its body. Such methods are themselves termed 'abstract'. Where a class contains an

abstract method it cannot be instantiated, since it is not specified what should happen were the method to be called.

An 'interface' is a class which has only abstract methods. However, such a class is declared not with the 'class' keyword but the 'interface' keyword. Above we stated that a C# class can only inherit from one 'base' class; but this ignores interfaces. A class can inherit from any number of interfaces.

Nested Classes

Classes are usually specified independently of each other. But it is possible for one class to be specified within another's specification. In this case, the latter class is termed a nested class.

Structs

A struct is a user-defined value type. It is declared in a very similar way to a class, except that it can't inherit from any class, nor can any class inherit from it (as mentioned previously, however, all value types do inherit from System.object). The following example shows a partial declaration for a 'Coordinate' struct:

```
1.  struct Coordinate
2.  {
3.      public int x;
4.      public int y;
5.
6.      public Coordinate(int x, int y)
7.      {
8.          this.x = x;
9.          this.y = y;
10.     }
11. }
```

Given the above, one could initialise a Coordinate type in a familiar way, using code like:

```
Coordinate c = new Coordinate(10, 2);
```

Note that if a variable of a struct type is declared without being given an explicit value, eg:

```
Coordinate c2 ;
```

it does not equate to 'null' (this being the default value for reference types, rather than value types). Instead, the variable is initialised to a state where its fields have their default values. If these fields are basic value types, they will generally be set to zero. If these fields are reference types, they will be set to 'null'.

Because of this default initialisation behaviour, it is an error for a struct to be given a parameterless constructor (eg. one like 'public Coordinate()'). Also, where a struct does have a constructor, you should be sure to make assignments to all of the struct's fields within this constructor.

Namespaces

Namespaces can be thought of as collections of classes; they provide unique identifiers for types by placing them in an hierarchical structure.

To illustrate the use of namespaces: suppose that two different C# developers come up with a class called 'bank', one relating to fiscal institutions and the other relating to riversides. In a programming environment containing both classes, there is a need to distinguish one from the other, and this is achieved by placing them within different namespaces. For example, the former class could be placed within the 'fiscal' namespace, say, becoming `fiscal.bank`, whereas the latter could be placed within the 'river' namespace becoming `river.bank`. (Note that C# does not include Java's direct link between the namespace hierarchy and the file structure hierarchy).

Most classes depend upon the existence of other classes - for instance, they may specify contained types. It is possible in the specification always to write each class' full namespace, but these are often too long for it to be worthwhile. To take an example at random, the following is the fully qualified name of a class in the .NET framework relating to a particular type of cryptographic algorithm:

```
System.Security.Cryptography.AsymmetricAlgorithm
```

This problem is addressed by the use of the 'using' keyword, placed at the very top of the class specification. For instance, in a class specification including the phrase

```
using System.Security.Crypography;
```

one could write refer to the above class simply using its class name

```
AsymmetricAlgorithm
```

Alternatively, one could specify an alias for the the namespace, eg

```
using myAlias = System.Security.Crypography;
```

and then refer to the class with

```
myAlias.AsymmetricAlgorithm
```

One specifies a namespace for one's own classes using the 'namespace' keyword. For instance, the following code states that the class 'Adder' is in the namespace `fred.math`.

```
1. namespace fred
2. {
3.     namespace math
4.     {
5.         public class Adder
6.         {
7.             // insert code here
8.         }
9.     }
```

10.

Alternatively, and more simply, one write the above as:

```
1. namespace fred.math
2. {
3.     public class Adder
4.     {
5.         // insert code here
6.     }
7.
```

Lesson 12: Class Declaration

Class declarations can have up to four different parts, surrounding the 'class' keyword:

attributes class-modifiers class class-base class-body

The class-body element specifies type members. The following is an example of a very simple class declaration, the class body being the lines following line 1:

```
1.    public class Shape
2.    {
3.        // class-body
4.    }
```

We now consider the other parts of a class declaration.

Attributes

Attributes can be posted at the front of a class declaration. These comprise user-defined 'meta-data' about the class; information which can be brought out at runtime. The example given in the C# language reference is this: one might define a 'HelpAttribute' attribute in order to map classes to their documentation. Attributes are covered in more detail in [lesson 18](#).

Class Modifiers

There are seven different - optional - class modifiers. Four of these - 'public', 'internal', 'protected' and 'private' - are used to specify the access levels of the types defined by the classes. The following five different access levels can be specified with these four modifiers:

public

The 'public' keyword identifies a type as fully accessible to all other types. This is the implicit accessibility of enumerator members ([lesson 7](#)) and interface members ([lesson 11](#)).

internal

If a class is declared as 'internal', the type it defines is accessible only to types within the same assembly (a self-contained 'unit of packaging' containing code, metadata etc.). This is the default access level of non-nested classes.

protected

If a class is declared as 'protected', its type is accessible by a containing type and any type that inherits from this containing type. This modifier should only be used for internal classes (ie. classes declared within other classes).

protected internal

The permissions allowed by this access level are those allowed by the 'protected' level plus those allowed by the 'internal' level. The access level is thus more liberal than its parts taken individually. This modifier should only be used for internal classes (ie. classes declared within other classes).

private

Where a class is declared as 'private', access to the type it defines is limited to a containing type only. This modifier should only be used for internal classes (ie. classes declared within other classes).

We now turn to the final three class modifiers:

new

The 'new' keyword can be used for 'nested' classes. A nested class is one that is defined in the body of another class; it is in most ways identical to a class defined in the normal way, but its access level cannot be more liberal than that of the class in which it is defined. A nested class should be declared using the 'new' keyword just in case it has the same name as (and thus overrides) an inherited type.

abstract

A class declared as 'abstract' cannot itself be instanced - it is designed only to be a base class for inheritance.

sealed

A class declared as 'sealed' cannot be inherited from.

Class Base

The 'class base' part of the class declaration specifies the name of the class and any classes that it inherits from.

As we noted previously, classes can inherit from just one base class and any number of interfaces. The classes to be inherited from are named following a colon after the class's own name (with any base class preceding any interfaces). The following line declares a public class called 'DrawingRectangle' which inherits from the base class 'Rectangle' and the interface 'Drawing':

```
public class DrawingRectangle : Rectangle, Drawing
```

Interface Declarations

Interfaces (described in [Lesson 13](#)) are declared in much the same way as standard classes, except that they use the keyword 'interface' in place of the keyword 'class'. For instance:

```
public interface Drawing
```

The other important difference is that the class modifiers 'abstract' and 'sealed' cannot be used with interface declarations (it would be unnecessary to use the former and illegitimate to use the latter).

Lesson 13: Introducing Methods

Methods are operations associated with types. To provide a type with methods is to give it some useful functionality. Often this functionality is made generally available, so that it can be utilised by other types

To take a simple example, suppose that we have an 'Arithmetic' class, whose purpose is to provide arithmetic operations. One simple method this class could have is the 'addTwoIntegers' method, whose job is to support the operation of adding two integers. To make use of this functionality, a piece of code would 'call' the method by 'passing' it two integers. The method would then 'return' their sum. Such a piece of code might look like this:

```
int sum = Arithmetic.addTwoIntegers(4,7);
```

A method declaration, specified within a class declaration, comprises a method-head and a method-body. The method-head is made up of the following elements (square brackets enclose those which are optional).

```
[attributes] [method-modifiers] return-type method-name ([ formal-  
parameter-list] )
```

In the case of the example method, its method head could look something like this:

```
[Description("a pointless method")] public static int addTwoIntegers(int a, int  
b)
```

Attributes

Method attributes work in a similar way to that briefly described for classes, see [lesson 18](#). We do not in this tutorial try to cover the different types of attributes that there are.

Method Modifiers

There are ten method modifiers that can be used. Four of these are the access modifiers that can be used in class declarations (see [lesson 12](#)). These four work analogously to the way they work in class declarations. The others are the following:

abstract

Abstract methods are discussed in [lesson 11](#)

static

The 'static' modifier declares a method to be a class method.

The methods (as well as the enumerators, properties and variables) specified in a class can be associated either with the class's instances (ie. the reference types it specifies) or with the class itself. These methods are called, respectively, 'instance methods' and 'class methods'. Class methods, declared with the 'static' modifier, can be called even when there exists no current instances of the class.

There is no equivalent modifier for instance methods, since methods are instance methods just in case their declaration does not include the word 'static'.

new, virtual, override

These modifiers concern the inheritance of methods from super- to sub-classes. They are covered in [lesson 14](#)

extern

Methods which are given as 'extern' are defined externally, using a language other than C#. We shall not go into the process involved in defining such methods.

Formal Parameters

A method's parameters are the types that get passed to it when the method is called. The list of parameters begins by specifying zero or more 'fixed parameters', and it may finish by specifying a single parameter-array. This latter element - declared using the 'params' keyword - means that it is possible to pass an arbitrary number of types to a single method. An example is given later in the lesson.

Fixed parameter specifications can have either two or three parts (ignoring attributes). The first, optional modifier can be either 'ref' or 'out'. The second part of the specification specifies the parameter's type, and the third part its name. Examples of these different elements can be seen in the illustrative code in the sections below.

[Modifier] parameter-type parameter-identifier

Parameter Passing

passing by value

The parameter modifiers 'ref' and 'out' relate to how the parameter is passed into the method. Where neither of these modifiers is used, the parameter is passed in 'by value'. In this case, when the method is called the value given is copied to the variable specified in the method declaration. The following example illustrates this point; note that the change made to variable b in the body of the 'change' method doesn't result in a change to the variable a used to invoke the method.

```
1.  public static void Main()
2.  {
3.      int a = 0;
4.      change(a); // following this method invocation, a
                    equals 0
5.  }
6.
7.  public static void change(int b)
8.  {
9.      b = 5;
10. }
```

In this example, it was a value type that was passed 'by value'. But reference types can also be passed 'by value'. As we saw previously, the immediate value held by a reference type variable is actually a memory address. So when this variable is passed 'by value', the memory address is copied to the variable specified in the method head. But of course, because the two variables will hold the same memory address, any changes made within the method body to the object located at that memory address will be reflected outside the method (although this doesn't apply for immutable reference types like strings, which act more like value types - see [lesson 4](#)).

passing by reference

In C# we can pass variables into methods 'by reference'. Where a variable is passed by reference, the 'ref' modifier must be used both in the method head and the method invocation (illustrated by the next code block).

Passing by reference is most obviously useful in cases where we want to treat a value type like a reference type. For instance, the method call in the following code does change the value of the variable a passed into the 'change' method.

```
1.    public static void Main()
2.    {
3.        int a = 0;
4.        change(ref a); // following this method invocation,
                        a==5
5.    }
6.
7.    public static void change (ref int b)
8.    {
9.        b = 5;
10.   }
```

'output' parameters

Where a method parameter is defined (and invoked) using the 'out' modifier, it is passed by reference. The difference between the 'out' and the 'ref' modifier is this: a parameter modified by the 'out' keyword need not be assigned a value before being passed into the method, but must be assigned a value in the method.

The reason that one might use output parameters is to return multiple values from a method. For instance, in the following code an integer and a boolean is passed to the 'change' method. This method sets the boolean to indicate whether or not the integer is greater than 0, and returns the value of the integer doubled.

```
1.    public static void Main()
2.    {
3.        bool b;
```

```

4.      int c = change(5, out b);
5.    }
6.
7.    public static int change (int a, out bool b)
8.    {
9.        b=false;
10.       if (a>0)
11.           b=true;
12.       return (2*a);
13.    }

```

The params modifier

One can pass an arbitrary number of types to a method by declaring a parameter array with the 'params' modifier. Note, though, that it is not necessary to place these additional types into an array before calling the method - they can simply be listed in the method invocation (see the next code block for an example).

Types passed as 'params' are all passed by value. The following code gives an example of the use of the 'params' modifier; the method called ignores the first type passed to it (a double) and returns the sum of all (an arbitrary number of) the integer values passed to it.

```

1.    public static void Main()
2.    {
3.        double a = 1;
4.        int b = 2;
5.        int c = 3;
6.        int d = totalIgnoreFirst(a, b, c);
7.    }
8.
9.    public static int totalIgnoreFirst(double a, params int[]
    intArr)
10.   {
11.       int sum = 0;
12.       for (int i=0; i < intArr.Length; i++)
13.           sum += intArr[i];
14.       return sum;
15.   }

```

Return Type

Methods can either return a type or not. A method that doesn't return a type must give its return type as 'void'. A method that does return a type must name the type returned.

A method will stop and return a value if it reaches a 'return' statement at any point in its execution. The type returned is given at the end of such a return statement; its type must correspond with that specified in the method declaration. The following piece of code illustrates this point.

```
1.    public static int exampleMethod()  
2.    {  
3.        int i =0;  
4.        // process i  
5.  
6.        return i;  
7.    }
```

Method Overloading

Each method has a signature. This comprises the method's name and its parameters (excepting their names), but not the method's return type. In the following method header, the elements making up the method's signature are emphasised - note also that the 'params' keyword is not included in the signature.

```
public static int myMethod(int a, ref double b, out bool c, params int[] d)
```

The importance of the signature is that no class is allowed to contain two methods with the same signature. Since the signature takes in more than the method name, however, it is possible for one class to have methods sharing a name. For example, a class with the method whose header is given above might also contain a method with the header:

```
public static int myMethod(int a, ref double b)
```

Note, however, that since neither its return type nor the params keyword are part of a method's signature this class could not also contain a method with the header:

```
public static void myMethod(int e, ref double f, out bool g, int[] h)
```

Lesson 14: Polymorphism (Inherited Methods)

As we have seen previously ([lesson 11](#)), classes take on the methods of the classes from which they inherit. In some cases, however, a class may want to 'overwrite' such a method. C# supports two different ways of method overwriting - 'hiding' or 'overriding'. Note that the term 'overwrite' is a term we have devised to cover both hiding and overriding.

Method overwriting makes use of the following three method-head keywords (see [lesson 13](#)):

new, virtual, override

The main difference between hiding and overriding relates to the choice of which method to call where the declared class of a variable is different to the run-time class of the object it references. This point is explained further below.

Method Overriding

Suppose that we define a Square class which inherits from a Rectangle class (a square being a special case of a rectangle). Each of these classes also specifies a 'getArea' instance method, returning the area of the given instance.

For the Square class to 'override' the Rectangle class' getArea method, the Rectangle class' method must have first declared that it is happy to be overridden. One way in which it can do this is with the 'virtual' keyword. So, for instance, the Rectangle class' getArea method might be specified like this:

```
1.    public virtual double getArea()  
2.    {  
3.        return length * width;  
4.    }
```

To override this method the Square class would then specify the overriding method with the 'override' keyword. For example:

```
1.    public override double getArea()  
2.    {  
3.        return length * length;  
4.    }
```

Note that for one method to override another, the overridden method must not be static, and it must be declared as either 'virtual', 'abstract' or 'override'. Furthermore, the access modifiers for each method must be the same.

The major implication of the specifications above is that if we construct a new Square instance and then call its 'getArea' method, the method actually called will be the Square instance's getArea method. So, for instance, if we run the following code:

```
Square      sq      =      new      Square(5);  
double area = sq.getArea();
```

then the `getArea` method called on the second line will be the method defined in the `Square` class.

There is, however, a more subtle point. To show this, suppose that we declare two variables in the following way:

```
Square      sq      =      new      Square(4);  
Rectangle r = sq;
```

Here variable `r` refers to `sq` as a `Rectangle` instance (possible because the `Square` class derives from the `Rectangle` class). We can now raise the question: if we run the following code

```
double area = r.getArea();
```

then which `getArea` method is actually called - the `Square` class method or the `Rectangle` class method?

The answer in this case is that the `Square` class method would still be called. Because the `Square` class' `getArea` method 'overrides' the corresponding method in the `Rectangle` class, calls to this method on a `Square` instance always 'slide through' to the overriding method.

Method Hiding

Where one method 'hides' another, the hidden method does not need to be declared with any special keyword. Instead, the hiding method just declares itself as 'new'. So, where the `Square` class hides the `Rectangle` class' `getArea` method, the two methods might just be written thus:

```
1.    public double getArea() // in Rectangle  
2.    {  
3.        return length * width;  
4.    }
```

```
1.    public new double getArea() // in Square  
2.    {  
3.        return length * length;  
4.    }
```

Note that a method can 'hide' another one without the access modifiers of these methods being the same. So, for instance, the `Square`'s `getArea` method could be declared as `private`, viz:

```
1.    private new double getArea()  
2.    {  
3.        return length * length;
```

```
4.    }
```

This leads us to an important point. A 'new' method only hides a super-class method with a scope defined by its access modifier. Specifically, where the access level of the hiding method is 'private', as in the method just described, this method only hides the super-class method for the particular class in which it is defined.

To make this point more concrete, suppose that we introduced a further class, `SpecialSquare`, which inherits from `Square`. Suppose further that `SpecialSquare` does not overwrite the `getArea` method. In this case, because `Square`'s `getArea` method is defined as private, `SpecialSquare` inherits its `getArea` method directly from the `Rectangle` class (where the `getArea` method is public).

The final point to note about method hiding is that method calls do not always 'slide through' in the way that they do with virtual methods. So, if we declare two variables thus:

```
Square      sq      =      new      Square(4);  
Rectangle r = sq;
```

then run the code

```
double area = r.getArea();
```

the `getArea` method run will be that defined in the `Rectangle` class, not the `Square` class.

Lesson 15: Constants, Fields, Properties and Indexers

Fields

Fields are variables associated with either classes or instances of classes. There are seven modifiers which can be used in their declarations. These include the four access modifiers 'public', 'protected', 'internal' and 'private' (discussed in [lesson 12](#)) and the 'new' keyword (discussed in [lesson 14](#)). The two remaining modifiers are:

static

By default, fields are associated with class instances. Use of the 'static' keyword, however, associates a field with a class. Where a field is declared in this way, the field will only ever have one value, regardless of the number of instances of the class in existence at any time.

readonly

Where a field is readonly, its value can be set only in a class declaration (for static fields) or in a constructor (for non-static fields).

Note that a field declaration can involve multiple fields, as in the following line of code

```
public static int a = 1, b, c = 2;
```

which is equivalent to

```
public          static          int          a          =          1;
public          static          int          b;
public static int c = 2;
```

Constants

Constants are unchanging types, associated with classes, that are accessible at compile time. Because of this latter fact, constants can only be value types rather than reference types. Constant declarations take the 'const' keyword (not 'static', even though they are associated with classes), and the five modifiers 'public', 'protected', 'internal', 'private' and 'new'.

The following is a simple constant declaration, although multiple constants can be simultaneously declared.

```
public const int area = 4;
```

Properties

Properties can be thought of as 'virtual' fields. From the outside, a class' property looks just like a field. But from the inside, the property is generated using the actual class fields.

Property declarations take just those modifiers taken by methods (see [lesson 13](#)) Unlike languages like Java, C# provides dedicated support for accession and mutation of these properties. Suppose, for instance, that a type contains an internal field called 'age'. With the

following code one could specify a property Age, providing accessors and mutators to this internal field.

```
1.  public int Age
2.  {
3.      get
4.      {
5.          return this.age;
6.      }
7.      set
8.      {
9.          this.age = value;
10.     }
11. }
```

Notice that the term 'value' is used in the above piece of code. This variable always holds the value passed to the 'set' block. For instance, the execution of the following line of code (assuming the appropriate class instance) would automatically set 'value' in the 'set' block to 4.

```
person.Age = 4;
```

This property Age can be described as 'read-write' since it can be both read from and written to. To make a property 'write-only' one simply does not specify a 'get' block; to make it 'read-only' one does not specify a 'set' block. The following piece of code demonstrates the read-only property 'Adult':

```
1.  public bool Adult
2.  {
3.      get
4.      {
5.          if (this.age<18)
6.              return false;
7.          else
8.              return true;
9.      }
10. }
```

Indexers

If properties are 'virtual fields', indexers are more like 'virtual arrays'. They allow a class to emulate an array, where the elements of this array are actually dynamically generated by function calls.

The following piece of code defines a class to hold a list of runners in an athletics race. The runners are held in lane order, and an indexer is exposed which allows the list to be both read from and written to. The indexer deals gracefully with cases in which the lane number passed to it is either too high or too low.

```
1.  class RaceDetails
2.  {
3.      private string[] lanes;
4.
5.      public RaceDetails()
6.      {
7.          this.lanes = new string[8];
8.      }
9.
10.     public string this[int i]
11.     {
12.         get
13.         {
14.             return (i >= 0 && i < 8) ? this.lanes[i] : "error";
15.         }
16.
17.         set
18.         {
19.             if (i >= 0 && i < 8) this.lanes[i] = value;
20.         }
21.     }
22. }
```

The following simple code illustrates use being made of the class just defined. The name of the person in the race's first lane is set, and then this name is sent to a console window.

```
1.  RaceDetails rd = new RaceDetails();
2.  rd[1] = "fred";
3.  Console.WriteLine("Lane One : " + rd[1]);
```

As can be seen from the example, an indexer is defined in a similar way to a property. One important difference is in the indexer's signature; the word 'this' is used in place of a name, and after this word indexing elements are provided.

Indexers aren't differentiated by name, and a class cannot declare two indexers with the same signature. However, this does not entail that a class is limited to just one indexer. Different indexers can have different types and numbers of indexing elements (these being equivalent to method parameters, except that each indexer must have at least one indexing element, and the 'ref' and 'out' modifiers cannot be used).

Because indexing elements are not limited to integers, the original description of indexers as 'virtual arrays' actually rather undersells them. For example, where the indexing elements include strings, indexers present themselves more like hash tables.

The following code shows an implementation for the RaceDetails class of an indexer whose indexing element is a string. Using this indexer it is possible to refer to a lane using the name of the person currently filling that lane.

```
1.  public string this[string s]
2.  {
3.      get
4.      {
5.          int laneNum = getCorrespondingLane(s);
6.          return (laneNum<0) ? "error" : this.lanes[laneNum];
7.      }
8.
9.      set
10.     {
11.         int laneNum = getCorrespondingLane(s);
12.         if (laneNum>=0) this.lanes[laneNum] = value;
13.     }
14. }
15.
16. private int getCorrespondingLane(string myName)
17. {
18.     for (int x=0; x<lanes.Length; x++)
19.     {
20.         if (myName==lanes[x]) return x;
21.     }
22.     return -1;
23. }
```

The following piece of code gives an example of the kind of use one might make of this string indexer.

```
1.  rd["fred"] = "jill";
```

Lesson 16: Delegates and Events

Delegates are reference types which allow indirect calls to methods ([lesson 13](#)). A delegate instance holds references to some number of methods, and by invoking the delegate one causes all of these methods to be called. The usefulness of delegates lies in the fact that the functions which invoke them are blind to the underlying methods they thereby cause to run (see, for instance, the discussion of events, below).

From this brief description, it can be seen that delegates are functionally rather similar to C++'s 'function pointers'. However, it is important to bear in mind two main differences. Firstly, delegates are reference types rather than value types (for the difference see [lesson 4](#)). Secondly, some single delegates can reference multiple methods

Delegate Declaration and Instantiation

Delegates can be specified on their own in a namespace, or else can be specified within another class (the examples below all show the latter). In each case, the declaration specifies a new class, which inherits from `System.MulticastDelegate`.

Each delegate is limited to referencing methods of a particular kind only. The type is indicated by the delegate declaration - the input parameters and return type given in the delegate declaration must be shared by the methods its delegate instances reference. To illustrate this: a delegate specified as below can be used to refer only to methods which have a single `String` input and no return value:

```
public delegate void Print (String s);
```

Suppose, for instance, that a class contains the following method:

```
1.    public void realMethod (String myString)
2.    {
3.        // method code
4.    }
```

Another method in this class could then instantiate the 'Print' delegate in the following way, so that it holds a reference to 'realMethod':

```
Print delegateVariable = new Print(realMethod);
```

We can note two important points about this example. Firstly, the unqualified method passed to the delegate constructor is implicitly recognised as a method of the instance passing it. That is, the code is equivalent to:

```
Print delegateVariable = new Print(this.realMethod);
```

We can, however, in the same way pass to the delegate constructor the methods of other class instances, or even static class methods. In the case of the former, the instance must exist at the time the method reference is passed. In the case of the latter (exemplified below), the class need never be instantiated.

```
Print delegateVariable = new Print(ExampleClass.exampleMethod);
```

The second thing to note about the example is that all delegates can be constructed in this fashion, to create a delegate instance which refers to a single method. However, as we noted before, some delegates - termed 'multicast delegates' - can simultaneously reference multiple methods. These delegates must - like our Print delegate - specify a 'void' return type.

One manipulates the references of multicast delegates by using addition and subtraction operators (although delegates are in fact immutable reference types - for explanation of the apparent contradiction see the discussion of strings in [Lesson 4](#)). The following code gives some examples:

```
1.    Print s = null;
2.    s = s + new Print (realMethod);
3.    s += new Print (otherRealMethod);
```

The - and -= operators are used in the same way to remove method references from a delegate.

The following code gives an example of the use of delegates. In the Main method, the Print delegate is instantiated twice, taking different methods. These Print delegates are then passed to the Display method, which by invoking the Print delegate causes the method it holds to run. As an exercise, you could try rewriting the code to make Print a multicast delegate.

```
1.    using System;
2.    using System.IO;
3.
4.    public class DelegateTest
5.    {
6.        public delegate void Print (String s);
7.
8.        public static void Main()
9.        {
10.            Print s = new Print (toConsole);
11.            Print v = new Print (toFile);
12.            Display (s);
13.            Display (v);
14.        }
15.
16.        public static void toConsole (String str)
17.        {
18.            Console.WriteLine(str);
19.        }
20.
```

```

21.    public static void toFile (String s)
22.    {
23.        File f = new File("fred.txt");
24.        StreamWriter fileOut = f.CreateText();
25.        fileOut.WriteLine(s);
26.        fileOut.Flush();
27.        fileOut.Close();
28.    }
29.
30.    public static void Display(Print pMethod)
31.    {
32.        pMethod("This should be displayed in the console");
33.    }
34. }

```

Events

To recap: in object-oriented languages, objects expose encapsulated functions called methods. Methods are encapsulated functions which run when they are invoked.

Sometimes, however, we think of the process of method invocation more grandly. In such a case, the method invocation is termed an 'event', and the running of the method is the 'handling' of the event. An archetypal example of an event is a user's selection of a button on a graphical user interface; this action may trigger a number of methods to 'handle' it.

What distinguishes events from other method invocations is not, however, that they must be generated externally. Any internal change in the state of a program can be used as an event. Rather, what distinguishes events is that they are backed by a particular 'subscription-notification' model. An arbitrary class must be able to 'subscribe to' (or declare its interest in) a particular event, and then receive a 'notification' (ie. have one of its methods run) whenever the event occurs.

Delegates - in particular multicast delegates - are essential in realizing this subscription-notification model. The following example describes how Class 2 subscribes to an event issued by Class 1.

1. Class 1 is an issuer of E-events. It maintains a public multicast delegate D.
2. Class 2 wants to respond to E-events with its event-handling method M. It therefore adds onto D a reference to M.
3. When Class 1 wants to issue an E-event, it simply calls D. This invokes all of the methods which have subscribed to the event, including M.

The 'event' keyword is used to declare a particular multicast delegate (in fact, it is usual in the literature to just identify the event with this delegate). The code below shows a class `EventIssuer`, which maintains an event field `myEvent`. We could instead have declared the event to be a property instead of a field (for the difference between these see [lesson 15](#)).

When the method `issueEvent` is called, the event `myEvent` is called, sending out both a reference to its parent object and arbitrary arguments.

```
1.  public class EventIssuer
2.  {
3.      public delegate void EventDelegate(object from,
4.      EventArgs args);
5.
6.      public event EventDelegate myEvent;
7.
8.      public void issueEvent(EventArgs args)
9.      {
10.         myEvent(this, args);
11.     }
12. }
```

A class which wanted to handle the events issued by an `EventIssuer ei` with its method `handleEvents` would then subscribe to these events with the code:

```
ei.myEvent += new EventIssuer.EventDelegate(handleEvents);
```

Lesson 17: Exceptions

The exception handling in C#, and Java is quite similar. However, C# follows C++ in allowing the author to ignore more of the exceptions that might be thrown (an exception which is thrown but not caught will halt the program and may throw up a dialogue box).

To catch a particular type of exception in a piece of code, you have to first wrap it in a 'try' block and then specify a 'catch' block matching that type of exception. When an exception occurs in code within the 'try' block, the code execution moves to the end of the try box and looks for an appropriate exception handler. For instance, the following piece of code demonstrates catching an exception specifically generated by division by zero:

```
1.    try
2.    {
3.        res = (num / 0);
4.    }
5.    catch (System.DivideByZeroException e)
6.    {
7.        Console.WriteLine("Error: an attempt to divide by
8.        zero");
9.    }
```

You can specify multiple catch blocks (following each other), to catch different types of exception. A complication results, however, from the fact that exceptions form an object hierarchy, so a particular exception might match more than one catch box. What you have to do here is put catch boxes for the more specific exceptions before those for the more general exceptions. At most one catch box will be triggered by an exception, and this will be the first (and thus more specific) catch box reached.

Following the last 'catch' box you can also include a 'finally' box. This code is guaranteed to run whether or not an exception is generated. It is especially useful for cleanup code where this would be skipped in the 'try' box following an exception being thrown.

Where an exception is not caught by any of the subsequent 'catch' boxes, the exception is thrown upwards to the code which called the method in which the exception occurred (note that in C# the methods do not declare what exceptions they are throwing). This exception will keep on bubbling upwards until it is either caught by some exception handling in the code, or until it can go no further and causes the program to halt.

Note that the exceptions a program throws need not be limited to those automatically generated. A program can throw exceptions - including customised exceptions - whenever it wishes, using the 'throw' command. The code below gives examples of all the statements discussed above, with the 'getException' method showing how to throw an exception.

```
1.    using System;
2.    public class ExceptionDemo
3.    {
```

```
4.     public static void Main ()
5.     {
6.         try
7.         {
8.             getException();
9.         }
10.        catch (Exception e)
11.        {
12.            Console.WriteLine("We got an exception");
13.        }
14.        finally
15.        {
16.            Console.WriteLine("The end of the program");
17.        }
18.    }
19.
20.    public static void getException()
21.    {
22.        throw new Exception();
23.    }
24. }
```

Lesson 18: Using the C# Compiler

As we have noted earlier, C# classes are compiled in the first place to the Common Language Runtime Intermediate Language (IL). And as shown in [lesson 3](#), one compiles these classes using the command-line command

```
csc file.cs
```

Where the required classes are held in more than one file, these should be listed, separated by spaces, as in:

```
csc file1.cs file2.cs
```

Broadly speaking, one can compile C# classes into either executable files or dynamic link library - DLL - files (see the /t switch in the table below). An executable file is one that contains a runnable program (note that in the classes compiled to an executable file, there should be only one 'Main' method). A .NET dynamic link library just collects together an assembly of classes, which can then be instantiated and utilised by programs in the course of their running.

If your classes reference external classes, the C# compiler must be able to locate them. By default, the compiler only looks within the 'mscorlib.dll' assembly, which supports the basic Common Language Runtime functions. To point the compiler in the direction of other classes, use the /r switch described in the table below (note that there is no equivalent to the Java approach of setting the environment variable CLASSPATH). To find the appropriate assembly for .NET framework classes, consult their documentation.

The following gives a list of the compiler switches we have found useful, but we would advise you to look further at the .NET documentation to see which other ones are available.

Compiler Switch	Description
/r:dll or /reference:dll eg. /r:System.xml.dll, System.Net.dll	This tells the C# compiler to include one or more of the .NET framework DLLs. As the standard library namespaces are not automatically referenced by the compiler, it is necessary to tell it which ones are needed. This switch also allows you to include your own DLLs.
/out: file eg. /out:fred.dll	Specifies the filename to which the compiled code is to be written.
/doc: file eg. /doc:doc.xml	Requests the production of xml documentation into the specified file (see lesson 19).
/t:type or /target:type eg. /t:exe - produce a console executable file (default) /t:library - produce a dll file /t:module - creates each	This is used to specify the type of output file produced

file into its own dll, eg. fred.cs will be converted to fred.dll /t:winexe - produce a windows executable file	
--	--

If you are regularly compiling a program and using a lot of switches in your program, we have found it useful to put the compile command in a batch file rather than writing out the entire command each time.

Preprocessor Directives

Preprocessor directives tags included within class specifications; they are used to give the compiler additional information about regions of code. The following example shows that you can specify areas of code to be compiled or not, depending upon the presence of a tag:

```
1.  /*
2.  Preprocessor Test
3.  */
4.
5.  #define DEBUG
6.  public class PreTest
7.  {
8.      public static void print (string s)
9.
10.         System.Console.WriteLine(s);
11.
12.
13.     public static void Main()
14.     {
15.         #if DEBUG
16.         print ("Hello");
17.         #endif
18.         print ("Andy");
19.     }
20. }
```

In the above, the #define statement on line 5 acts as a boolean: it sets the variable DEBUG to be 'defined' rather than 'undefined'. Because it is defined, the code on line 16 gets compiled. If we were to remove the statement on line 5, the compiler would effectively treat the code on line 16 as commented out.

The following gives a list of some of the available preprocessor directives.

Directive	Action
-----------	--------

#define symbol	Sets symbol to be 'defined' (true)
#undef symbol	Sets symbol to be 'undefined' (false)
#if symbol [operator symbol2]	The if statement evaluates the given expression. The possible operators can be ==, !=, &&, . If the expression evaluates to 'true', the code to the #else, #elif or #endif directive is compiled.
#else	Used in conjunction with the if statement.
#elif	Used in conjunction with the if statement as 'else-if'.
#endif	Ends the previous conditional directives
#warning text	The given text appears as a warning in the compiler output
#error text	The given text appears as an error in the compiler output
#line number[file]	Outputs a line number, and (optionally) a filename to the compiler output.
#region name	Marks the beginning of a region
#end region	Marks the ends of a region

Attributes

Attributes are used to give extra information to the .NET compiler. C# and the .NET framework have a few built-in attribute types, but it is also possible to create new ones by extending the System.Attribute class. Below we describe a few common uses of attributes.

It is possible to tell the compiler that a class is compliant with the .NET Common Language Specification (discussed in [lesson 1](#)) with code like the following:

```

1.    using System;
2.
3.    [CLSCompliant(true)]
4.    public class myClass
5.    {
6.        // class code
7.    }
```

Similar code can also be used to indicate that a class has been obsoleted.

Web services (mentioned in lesson 1) also make heavy use of attributes. Demonstrated by the example below, the attribute [WebMethod] is used to specify that a particular method is to be exposed as a web service.

```

1.    [ WebMethod ]
2.    public int Add(int num1, int num2)
3.    {
4.        return num1+num2;
```

5. }

Lesson 19: Code Documentation

The C# compiler supports the automatic creation of class documentation. Where the equivalent functionality for Java produces HTML, the C# documenter produces XML. This means that the C# documentation is not as immediately ready to use as the Java documentation. However, it does allow there to be different applications which can import and use the C# documentation in different ways. (Note: using Visual Studio you can also create HTML documentation, but we will not be covering this here).

Sadly, Microsoft did not bundle a basic documentation reader with C#.. Even worse, however, the documentation automatically produced by C# is rather less extensive than that produced by Java's javadoc tool. Indeed, as we note in the final section of this lesson, this XML documentation is so lacking that we have been driven to write an alternative documenter.

C# Documentation Comments

To document any element in a C# script, you precede the element with XML elements. Each of the lines comprising this documentary code should be marked off as comments using the following special comment indicator (you can compare this with the standard comment indicators in [Lesson 3](#))

```
///
```

The following code gives an example of how one can provide overview information about a class.

```
1.    /// <summary>
2.    /// The myClass class represents an arbitrary class
3.    /// </summary>
4.    public class myClass
```

You are at liberty to use any XML tags you wish to document the code – as long as they follow the XML syntax then the compiler will happily write them into the documentation. But Microsoft does provide a list of recommended XML elements for you to use. Some of these elements indicate the type of documentary information that is being given, and the compiler will validate certain aspects of these. Other elements are just used to give layout or formatting information.

The following lists describe the main documentation elements provided. Note that the content of each element should be written between its opening and closing tags, and some of the tags also take further attributes. In particular, the 'cref' attribute can supposedly be used in any element, but we have just used it in the cases where it seems particularly appropriate.

Tag(s)	Description
<summary>	- holds overview information about any documentable element.
<remarks>	- allows for expanded comments about any documentable element, following summary information.

Note: we still aren't sure if the descriptions of the tags above are correct. The following points describe the problem.

In favour of using the 'summary' and 'remarks' in the suggested way is the fact that Gunnarson, who helped create C#, sets things out in this way. It also correlates with the default behaviour of Visual Studio.NET, where 'summary' tags are given by default whenever you start documenting any element.

On the other hand, in the help files of the (production) version of .NET – v. 1.0.3705 – it explicitly states that the use of 'summary' is to hold overview information about a class member, whereas the use of 'remarks' is to hold overview information about a class. However, some of the examples given throughout these same help files conflicts with this advice - for example, in the description of the 'paramref' element, a class method is documented only with 'remarks'. Unfortunately, of course, this example also conflicts with what we say, since the example contains no 'summary' tag.

Basically, it's all been knocked together by a committee of rather befuddled monkeys. But the way we suggest is as good as any.

Tag(s)	Description
<param name="name">	- describes a parameter passed to a method. The compiler checks that the 'name' value matches an actual parameter in the code. Also, if you give documentation for one parameter value, the compiler will expect you to give documentation for them all.
<paramref name="name">	- identifies the mention of a parameter name within some other descriptive element, for instance within 'summary' tags. The idea is that this mentioned name can be styled differently from the surrounding text. The compiler checks that the 'name' value matches an actual parameter in the code.
<returns>	- describes the return value for a method. As the descriptive field is just free text there is no compiler checking.
<exceptions cref="type">	- describes an exception that may be thrown by a method. The 'cref' attribute refers to a type or field (such as System.Exception), and the compiler checks that this reference is available from the current compilation environment.
<permission cref="type">	- describes a permission requirement for a type or member. The cref attribute refers to a type or field (such as System.Security.PermissionSet), and the compiler checks that this reference is available from the current compilation environment.
<value>	- describes a class property.
<example>	- gives an example of the use of the referenced object (for example a class method). The 'example' element is often used with the following two elements.
<c>	- marks up a single phrase or line as a code example. Generally used in conjunction with the 'example' element.
<code>	- marks up multiple lines as code examples. Generally used in conjunction with the 'example' element.

<code><see cref="type"></code>	- used to identify a cross-reference in the documentation; designed to be used inline within a description element. The cref attribute refers to a type or field, and the compiler checks that this reference is available from the current compilation environment. and the see also tag is used in a separate section. This allows the documentation to create cross-references.
<code><seealso cref="type"></code>	- used to identify a cross-reference in the documentation; different from 'see' in that it is designed to stand alone. The cref attribute refers to a type or field, and the compiler checks that this reference is available from the current compilation environment.

The following elements are just used to provide layout information:

<code><para></code>	- used within descriptive tags like 'remarks', 'summary', etc. to wrap a single paragraph.
<code><list type = "bullet" "number" "table"></code>	- top level tags for a list, where this may be one of the three types shown. There are more elements associated with the list tag: the following code gives an example of these. <pre> <list type="table"> <listheader> <term>Animal</term> <description>Type</description> </listheader> <item> <term>monkey</term> <description>hairy</description> </item> <item> <term>pig</term> <description>bald</description> </item> </list> </pre>

Note - in relation to the example of the 'list' tag given above - that the v. 1.0.3705 help documentation for the enclosed 'item' tag talks about a 'text' element in place of the second 'description'. But this seems to be just wrong.

Generating C# Documentation

You tell the compiler to produce documentation when compiling by invoking it with the switch:

```
/doc:file
```

In this switch, file represents the name of the file that you want the documentation written to. As the documentation is generated in xml format, the file should have the extension .xml. So, for instance, to produce the documentation for a program in sys.cs file in a file named my.xml, we would use the command:

```
csc sys.cs /doc:my.xml
```

Those working with Visual Studio .NET should set the 'XML Documentation File' property in the 'Build' directory of the project's Configuration Properties.

Problems with the C# Documenter

The C# documenter is lacking in many different areas. Here are just a couple of problems that make the documenter hard to live with.

Firstly, if you don't document a method, the documenter just ignores it (or else throws a warning). This forces you to document all methods, no matter how simple and obvious they might be, which is annoying when the computer could easily do it for you.

Secondly, while the compiler checks the parameter types of methods, even to the extent of providing the fully qualified parameter name, it fails to do this with the return values of methods. The user has to manually insert the return type, even though the compiler could easily produce this.

There is a solution, however...

The Softsteel Documenter

Driven by documentation frustration, the Softsteel team (well, Andy really, but we all chipped in with biscuits), has produced a command-line documenter, written in C#. It's quite rusty at the moment, but is already a whole load better than the inbuilt documenter. We're working on improvements, but we're releasing it as free software under the GNU GPL in order to encourage community support.

The downloads for the documentation tool are now being handled by the download page <http://www.softsteel.co.uk/tutorials/cSharp/download.asp>

Lesson 20: Further References

As C# is finalized, and more resources built up around it, we shall be looking to add to this references page.

C# Language Reference

<http://msdn.microsoft.com/net/ecma/WD13Oct01.doc>

.NET Download

<http://www.msdn.microsoft.com/downloads/default.asp?url=/downloads/sample.asp?url=/msdn-files/027/000/976/msdncompositedoc.xml&frame=true>

General Portals

Microsoft Community website for .NET - the C# production team pages
<http://www.gotdotnet.com/team/csharp/>

A new portal "dedicated to furthering knowledge of and expertise in C# language development."
<http://www.csharp.org/>

Information, Links and other Resources for the C# Language
<http://www.csharp-station.com/>

C# articles, forum, etc.
<http://www.pune-csharp.com/>

A very extensive list of dot-net links
<http://www.dot42.com/>

The latest .Net Users Group in the UK, serving the north of England.
<http://www.dnugnorth.net/dnug/>

Collections of Articles

Working with C#
<http://www.msdn.microsoft.com/columns/>

.NET home page at Microsoft (the Dr GUI.NET columns are especially useful)
<http://www.msdn.microsoft.com/net/>

The O'Reilly C# page
<http://www.oreillynet.com/topics/dotnet/csharp.net>

Code-Project: useful code help
<http://www.codeproject.com/csharp/>

Codeguru
<http://codeguru.earthweb.com/csharp/index.shtml>

C# Discussion

GotDotNet message board

<http://www.gotdotnet.com/community/messageboard/MessageBoard.aspx?id=6>

Microsoft public newsgroup (web frontend)

<http://communities.microsoft.com/newsgroups/default.asp?icp=dotnet&slcid=us>

Codeguru discussion board

<http://codeguru.earthweb.com/cgi-bin/bbs/wt/wwwthreads.pl?action=list&Board=CSharp>

Open Source C# Projects

The Mono project is an attempt to create an open source implementation of the .NET Framework, including a C# compiler

<http://www.go-mono.com/>

The dotGNU project is trying to provide alternatives for all elements of the Microsoft .NET strategy, especially its Hailstorm application. It also includes a C# compiler.

<http://www.gnu.org/projects/dotgnu/>

Books on C#

We have restricted the following list to books covering the final release of C#. Because a lot of books have been released recently, and we have other things to do, we haven't read many of them ourselves. So many of the descriptions are culled from the books' marketing materials (indicated by the use of quotes). If you'd like to give us feedback on any of these books, which we can incorporate into the description, we'd love to hear it.

Clicking on the picture of a book will take you to that product at Amazon - either Amazon.co.uk for UK customers, or Amazon.com for international customers. If you consequently buy the book, we receive some money from Amazon. So if you were planning on buying a book anyway, and have found these lessons useful, this would be a nice way to support us.

The link to the book's publisher is to their pages covering that book. These pages tend to contain sample code, example chapters, errata, etc.

C# in a Nutshell - Drayton,Albahari and Neward



[\(UK\)](#)

Publisher: [O'Reilly](#)

The usual O'Reilly 'in a nutshell' fare. Professional, pretty comprehensive, providing crucial reference material as well as informed overview. Ideal for the programmer who already knows the basics of C# and needs reference material to hand. It doesn't cover all of the namespaces, however (for a list of those that are present, see the reviews on Amazon, where the editor has posted a message on this topic).



[\(Int\)](#)

Programming C# - Jesse Liberty



[\(UK\)](#)

Publisher: [O'Reilly](#)

Books published by O'Reilly tend to be precise and to the point, and this is no exception. The first part of the book covers C# syntax. The second - 'Programming with C#' - covers Windows Forms, ADO.NET, and ASP.NET. The third part covers a number of issues relating to C# and the .NET Framework (eg. threading).



[\(Int\)](#)

C# Essentials (2nd Edition) - Merrill, Drayton & Albahari



[\(UK\)](#)

Publisher: [O'Reilly](#)

"This book's compact format and terse presentation of key concepts serve as a roadmap to the online documentation included with the Microsoft .NET Framework SDK; the many examples provide much-needed context."



[\(Int\)](#)

The Complete Visual C# Programmer's Guide - Ozkir, et al



(UK)

Publisher: [Microgold Publishing \(link through to C# Corner\)](#)

This book was written by the authors of C# Corner, an online C# and .NET portal. According to its description: "The book is geared toward the intermediate programmer, but contains enough material to satisfy the advanced developer. The book starts by introducing the reader to the .NET environment and some basic C# program examples. The book then goes in depth on how to navigate the Visual Studio environment and introduces the reader to the C# language. From here the book dives into the .NET framework and how to take advantage of its capabilities using C#."



(Int)

Beginning C# - Wrox Author Team et al



(UK)

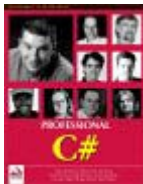
Publisher: [Wrox](#)

"Beginning C# provides a painless introduction to C# for beginners or relatively inexperienced programmers, who want to move to the .NET Framework from languages that don't support modern object-oriented programming techniques."



(Int)

Professional C# - Simon Robinson et al



(UK)

Publisher: [Wrox](#)

Wrox Press books tend to be very large, and to cover topics exhaustively from various different angles. This leads to some duplication of effort and patchiness of quality, but this is counterbalanced by the quantity of information.



(Int)

Professional C# Web Services: Building .NET Web - Thomas Palmer et al



(UK)

Publisher: [Wrox](#)

"This book is for anyone who is interested in building web services using the tools supplied with the .NET Framework, and wants to look at both of these technologies."



(Int)

C# Black Book - Telles



(UK)

Publisher: [Coriolis](#)

"Written by a Microsoft insider and expert programmer, this book contains comprehensive coverage of the C# programming language, compiler, and Common Library Runtime of the language. It contains an overview of the capabilities and syntax of the C# programming language. "



(Int)

C# Core Language Little Black Book - Wagner



(UK)

Publisher: [Coriolis](#)

"This book focuses on the core C# language. It covers the language syntax, provides samples, and shows you how to use C# with ASP.NET, WinForms, and Web Services. The book illustrates C# syntax and design strategies and how C# integrates with other languages--a real-world consideration. "



(Int)

C# and the .NET Platform - Andrew Troelsen



(UK)

Publisher: [Apress](#)

This book covers the C# syntax fairly quickly, before moving on to its main task of how to write .NET applications. It has been well received by the programming community.



(Int)

A Programmer's Introduction to C# - Eric Gunnerson



(UK)

Publisher: [Apress](#)

As Gunnerson was involved in the developing and testing of C#, the information in this book is guaranteed to be top quality. The organisation and presentation of the material is less perfect, but has improved from the first edition.



(Int)

C# Primer - A Practical Approach - Stanley Lippman



(UK)

Publisher: [Addison Wesley Longman, Inc.](#)

Introduces C# and then uses it to explore the .NET framework classes.



(Int)

C# Tips and Techniques - Charles Wright



(UK)

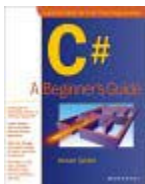
Publisher: [McGraw-Hill](#)

"Filled with hundreds of ready-to-run solutions you can use to build Web services and Windows applications"



(Int)

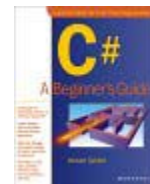
C#: A Beginner's Guide - Herb Schildt et al



(UK)

Publisher: [Osborne](#)

"See how C# relates to, and interacts with, the .NET Framework, and cover all the C# essentials - control statements, datatypes, arrays, strings, classes, objects, methods, and namespaces. Get full details on exception



(Int)

handling and the I/O system, and explore C#'s most innovative features, such as delegates, indexers, events, and properties."

C# Developer's Headstart - Michaelis & Spokas



[\(UK\)](#)

Publisher: [Osborne](#)

"Divided into six easy-to-navigate sections, C# Developer's Headstart is designed to deliver relevant information, tips, and advice to IT professionals and companies concerning adoption, integration, and ultimately migration to this powerful new language."



[\(Int\)](#)

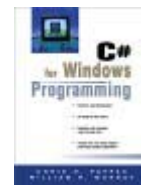
C# for Windows Programming - Pappas & Murray



[\(UK\)](#)

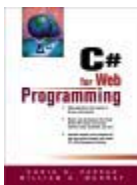
Publisher: [Prentice Hall](#)

"C# for Windows Programming gives you everything you need to get started in this next wave of Windows application programming."



[\(Int\)](#)

C# for Web Programming - Pappas & Murray



[\(UK\)](#)

Publisher: [Prentice Hall](#)

"Programmers with basic familiarity in C or C++ will find C# for Web Programming the best, quickest, and easiest way to learn C# development for the Web."



[\(Int\)](#)

C# Essentials - Pappas & Murray



[\(UK\)](#)

Publisher: [Prentice Hall](#)

"If you've worked with C or C++, C# Essentials will help you leverage the skills you've already developed - and if you haven't, it'll teach you C# from scratch."



[\(Int\)](#)

Application Development using C# and .NET - Stiefel & Oberg



[\(UK\)](#)

Publisher: [Prentice Hall](#)

"Build industrial-strength .NET applications with C#!"



[\(Int\)](#)

Introduction to C# using .NET - Oberg



Publisher: [Prentice Hall](#)

"C# hands-on. Your complete, step-by-step introduction to C# development for .NET."



[\(UK\)](#)

[\(Int\)](#)

C# Unleashed - Joseph Mayo



[\(UK\)](#)

Publisher: [Sams](#)

"the most basic elements of C# language syntax...object and component programming...several of the class libraries available...advanced topics for extreme performance and enterprise programming projects...garbage collection and the effects on a program., cross-language development, the common language runtime, versioning and assemblies...supplementary material on compiling programs, an overview of .NET Class Library components..."



[\(Int\)](#)

C# Primer Plus - Klaus Michelsen



[\(UK\)](#)

Publisher: [Sams](#)

"With C# Primer Plus's dual emphasis on C# as well as fundamental programming techniques, this friendly tutorial will soon make you a proficient C# programmer building Windows applications on the .NET platform."



[\(Int\)](#)

Sams Teach Yourself Web Programming in 21 Days - Syme & Aitken



[\(UK\)](#)

Publisher: [Sams](#)

"Learn how to how to use C# for Internet programming with the hands-on techniques and clear explanations. "



[\(Int\)](#)

Sams Teach Yourself C# in 21 Days - Bradley Jones



[\(UK\)](#)

Publisher: [Sams](#)

"This complete guide covers topics from basic program construction to intermediate level application engineering. Following "21 days" formula, this book is a three week intensive course for the beginning programmer who wishes to get started with this exiting new coding standard. "



[\(Int\)](#)

C# and the .NET framework - Powell & Weeks



[\(UK\)](#)

Publisher: [Sams](#)

"This book covers topics ranging from the general principles of .NET through the C# language and how its used in ASP.NET and Windows Forms. Written by programmers for programmers..."



[\(Int\)](#)

Presenting C# - Christophe Wille



[\(UK\)](#)

Publisher: [Sams](#)

"this book covers the C# language from the ground up with rich coding examples, and expert insight into issues you will need to understand to fully take advantage of the power of C#. Presenting C# assumes you are already familiar with key programming concepts such as variables, and programming constructs"



[\(Int\)](#)

C#.NET : Web Developer's Guide - Greg Hack



[\(UK\)](#)

Publisher: [Syngress](#)

"The focus of The C#.NET Web Developer's Guide is on providing you with code examples that will help you leverage the functionalities of the .NET Framework Class Libraries. One you have read this book, you will have covered the key concepts, libraries, and APIs of the .NET Framework that we feel will help you easily create new applications using C#."



[\(Int\)](#)

Programming Windows with C# - Charles Petzold



[\(UK\)](#)

Publisher: [Microsoft Press](#)

"In this .NET-ready Windows programming guide, the ... author shows you how to get the most out of Windows Forms—the next-generation Windows programming class library. You'll discover how to use C# to create dynamic user interfaces and graphical outputs for Windows applications."



[\(Int\)](#)

The Patchwork Book

It is common for publishers of programming books to provide sample chapters. What we have done in this section is to collect links to some of these sample chapters, the idea being to provide a kind of 'patchwork book'. As might be expected, the spread of topics covered is somewhat - ahem - patchy (heavy on introduction, since most sample chapters are introductions), but we hope that it will prove useful. As more books on C# come out, we shall look to add to this resource.

[1. Introduction \(i\)](#) (pdf)

- provides background information about the .NET framework, the Common Language Runtime, etc.
- from 'Application Development using C# and .NET', Stiefel & Oberg, Prentice Hall, 2002.

[2. Introduction \(ii\)](#) (html)

- a general introduction to the C# language (in six parts - use the links at the top of the page).

- from 'C# Primer Plus', Klaus Michelsen, Sams, 2001.

3. Introduction (iii) (pdf)

- getting started using C# in Visual Studio.NET
- from 'C# for Windows Programming', Pappas & Murray, Prentice Hall, 2002

4. Variables (html)

- value types and reference types, constants, etc. (in eleven parts - use the links at the top of the page)
- from 'Sams Teach Yourself C# in 21 Days', Bradley Jones, Sams, 2001'

5. Operators and Expressions (html)

- covers how to create expressions in C# using the range of available operators (in twelve parts - use the links at the top of the page)
- from 'C# Unleashed', Joseph Mayo, Sams, 2001

6. Advanced Classes (pdf)

- covers making indexers, overloading operators, delegates and events, and documenting classes.
- from 'C# and the .NET Framework', Andrew Troelsen, Apress Ltd., 2001

7. Interfaces (pdf)

- employing the interfaces built into the .NET framework in your classes.
- from 'Introduction to C# Using .NET', Oberg, Prentice Hall, 2002.

8. Attributes and Reflection (html)

- covers the use of metadata in programming in C#
- from 'Programming C#', Jesse Liberty, O'Reilly, 2001

9. Programming Windows forms with C# (html)

- an introduction to accessing Windows API forms with C#
- from 'Programming Windows with C#', Microsoft Press, 2001